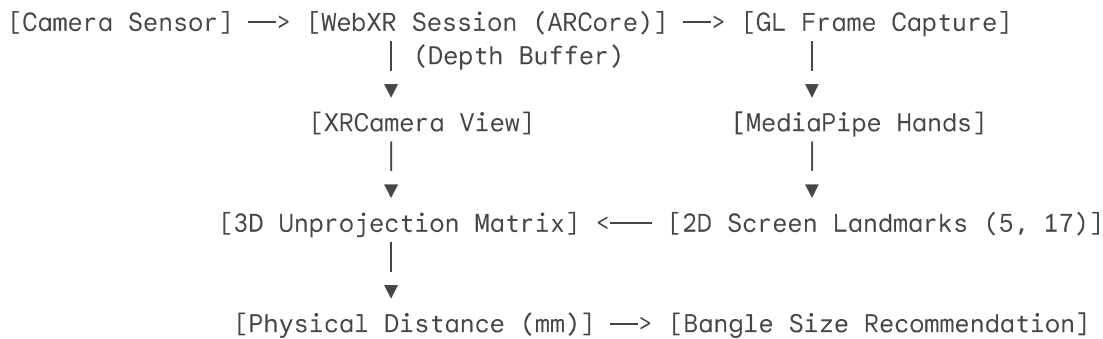


Technical Specification & Implementation Blueprint: WebXR-Powered Mobile Bangle Sizer (Android)

This document is designed as a step-by-step developer specification for a coding agent to construct, calibrate, and test a mobile-first, cardless bangle sizing web application utilizing **WebXR (with Depth-Sensing & Hit-Testing)** and **Google MediaPipe Hands** on Android Chrome.

1. Core Mathematical & Architectural Logic

A physical bangle must slide over the widest part of a compressed hand. Traditional computer vision struggles with distance scaling (the "depth problem"). By using WebXR, we retrieve real-world physical dimensions ($1.0 \text{ unit} = 1.0 \text{ meter}$) directly from the device's ARCore engine.



1.1 Sizing Strategy (Landmarks 5 and 17)

- **Landmark 5:** Base of the index finger (Metacarpophalangeal / MCP joint).
- **Landmark 17:** Base of the pinky finger (MCP joint).
- The physical distance d between these points on a tightly squeezed hand dictates the minimal inner diameter of the bangle.

1.2 Mathematical Projection (2D Screen Space to 3D View Space)

Given a 2D landmark point on the screen $P_{screen} = (u, v)$ where $u \in [0, \text{width}]$ and $v \in [0, \text{height}]$, we must compute its real-world 3D view-space coordinates $P_{view} = (X_{view}, Y_{view}, Z_{view})$.

1. Calculate Normalized Device Coordinates (NDC):

$$x_{ndc} = \frac{2u}{\text{width}} - 1$$

$$y_{ndc} = 1 - \frac{2v}{\text{height}}$$

$$z_{ndc} = 2d_{depth} - 1$$

Where d_{depth} is the physical depth value extracted from the WebXR CPU depth buffer (`getDepthInMeters(u, v)`) at the matching pixel coordinate.

2. **Retrieve WebXR Projection Matrices:** We pull the camera's Projection Matrix M_{proj} and its Inverse M_{proj}^{-1} from the active `XRView` .
3. **Compute 3D View-Space Position:** By applying the inverse projection matrix to the NDC coordinates:

$$\begin{bmatrix} X_{view} \\ Y_{view} \\ Z_{view} \\ W_{view} \end{bmatrix} = M_{proj}^{-1} \begin{bmatrix} x_{ndc} \\ y_{ndc} \\ z_{ndc} \\ 1 \end{bmatrix}$$

We then divide by the homogeneous coordinate W_{view} to retrieve physical meters:

$$X_{final} = \frac{X_{view}}{W_{view}}, \quad Y_{final} = \frac{Y_{view}}{W_{view}}, \quad Z_{final} = \frac{Z_{view}}{W_{view}}$$

4. **Calculate Euclidean Knuckle Width:** Once both Landmark 5 (P_1) and Landmark 17 (P_2) are projected into 3D view-space, the absolute hand width in millimeters is:

$$\text{Hand Width (mm)} = \sqrt{(X_{final,1} - X_{final,2})^2 + (Y_{final,1} - Y_{final,2})^2 + (Z_{final,1} - Z_{final,2})^2}$$

2. Environment & Testing Environment Setup

WebXR and camera permissions require strict security contexts. Testing cannot be done using raw file setups (`file://`).

1. Secure Port Forwarding:

- Run a local development server on your laptop (e.g., Node.js with Vite or Python's `http.server` on port `8080`).
- Connect your physical Android phone to your laptop via USB.
- Enable **Developer Options** and **USB Debugging** on the Android device.
- Open Chrome on your desktop and navigate to `chrome://inspect` .
- Enable **Port Forwarding**: Map port `8080` on your phone to `localhost:8080` on your laptop. This forces Chrome on Android to treat the connection as a secure origin (`http://localhost:8080`), bypassing HTTPS requirements for WebXR testing.

2. Device Requirements:

- Android device running Android 10 or newer.
- **Google Play Services for AR (ARCore)** installed.
- Google Chrome (v110+) configured as the default browser.

3. The Camera Resource Conflict Bypass (Critical Hack)

On mobile browsers, ARCore claims exclusive hardware control of the camera sensor. If you try to run `navigator.mediaDevices.getUserMedia()` while WebXR is active, the camera initialization will fail with a `NotReadableError` or instantly crash the WebXR session.

The Solution: WebGL Framebuffer Extraction

We must read the raw WebGL textures that ARCore is rendering on the screen during the `onXRFrame` loop, copy the pixels to an offscreen canvas, and feed that canvas directly into MediaPipe Hands.

```
// Step-by-step extraction workflow
let offscreenCanvas = document.createElement('canvas');
let offscreenCtx = offscreenCanvas.getContext('2d');

function extractWebXRFrame(gl, frame, session) {
  const baseLayer = session.renderState.baseLayer;
  gl.bindFramebuffer(gl.FRAMEBUFFER, baseLayer.framebuffer);

  // Set dimensions to match WebXR viewport
  const viewport = session.renderState.baseLayer.getViewport(frame.getViewerPose(
  offscreenCanvas.width = viewport.width;
  offscreenCanvas.height = viewport.height;

  // Extract pixels from the WebGL context
  const pixels = new Uint8Array(viewport.width * viewport.height * 4);
  gl.readPixels(
    viewport.x, viewport.y,
    viewport.width, viewport.height,
    gl.RGBA,
    gl.UNSIGNED_BYTE,
    pixels
  );

  // Write pixels back to 2D canvas context for MediaPipe input
  const imageData = offscreenCtx.createImageData(viewport.width, viewport.height)
  // Vertically flip the image since WebGL coordinates are inverted relative to s
  for (let y = 0; y < viewport.height; y++) {
    const srcRow = y * viewport.width * 4;
    const destRow = (viewport.height - 1 - y) * viewport.width * 4;
    imageData.data.set(pixels.subarray(srcRow, srcRow + viewport.width * 4), dest
  }
  offscreenCtx.putImageData(imageData, 0, 0);
  return offscreenCanvas;
}
```

4. WebXR Session & Depth Sensing Lifecycle

This lifecycle initializes the AR hardware with CPU-optimized depth configurations, which yields the highly accurate raw distance arrays we need.

```
let xrSession = null;
let xrRefSpace = null;
let xrDepthInfo = null;

async function startAR() {
  if (!navigator.xr) {
    showError("WebXR is not supported on this browser.");
    return;
  }

  const supported = await navigator.xr.isSessionSupported('immersive-ar');
```

```

if (!supported) {
  showError("Immersive AR is not supported on this device.");
  return;
}

try {
  // Request CPU-optimized depth-sensing and hit-test capability
  xrSession = await navigator.xr.requestSession('immersive-ar', {
    requiredFeatures: ['local', 'depth-sensing', 'hit-test'],
    depthSensing: {
      usagePreference: ['cpu-optimized'],
      dataFormatPreference: ['luminance-alpha']
    }
  });

  const canvas = document.createElement('canvas');
  const gl = canvas.getContext('webgl', { xrCompatible: true });
  xrSession.updateRenderState({ baseLayer: new XRWebGLLayer(xrSession, gl) });

  xrRefSpace = await xrSession.requestReferenceSpace('local');
  xrSession.requestAnimationFrame((time, frame) => onXRFrame(time, frame, gl));

} catch (err) {
  showError("AR Initialization failed: " + err.message);
}

function onXRFrame(time, frame, gl) {
  if (!xrSession) return;
  xrSession.requestAnimationFrame((t, f) => onXRFrame(t, f, gl));

  const pose = frame.getViewerPose(xrRefSpace);
  if (pose) {
    const view = pose.views[0];

    // Extract Depth Information map
    try {
      xrDepthInfo = frame.getDepthInformation(view);
    } catch (e) {
      console.warn("Depth information unavailable for this frame", e);
    }

    // Step 1: Capture frame pixels for computer vision
    const frameCanvas = extractWebXRFrame(gl, frame, xrSession);

    // Step 2: Pass frame to MediaPipe Hands (asynchronously)
    detectHandLandmarks(frameCanvas, view);
  }
}

```

5. MediaPipe Detection & Coordinate Translation Engine

To map the tracked hand landmarks back into the real world, write an async coordinate translator function.

```

// Load MediaPipe Hands via CDN scripts in your HTML head
// [https://cdn.jsdelivr.net/npm/@mediapipe/hands/hands.js](https://cdn.jsdelivr.net

```

```

let handsDetector = new Hands({
  locateFile: (file) => `https://cdn.jsdelivr.net/npm/@mediapipe/hands/${file}`
});
handsDetector.setOptions({
  maxNumHands: 1,
  modelComplexity: 1,
  minDetectionConfidence: 0.8,
  minTrackingConfidence: 0.8
});

handsDetector.onResults((results) => {
  if (results.multiHandLandmarks && results.multiHandLandmarks.length > 0) {
    processHandLandmarks(results.multiHandLandmarks[0]);
  } else {
    updateUIState("Searching for hand...");
  }
});

async function detectHandLandmarks(canvasElement, xrView) {
  // Store the active camera projection matrix and viewport
  activeProjectionMatrix = xrView.projectionMatrix;
  activeViewport = xrSession.renderState.baseLayer.getViewport(xrView);

  await handsDetector.send({ image: canvasElement });
}

function processHandLandmarks(landmarks) {
  // Landmark 5 (Index Knuckle) and 17 (Pinky Knuckle)
  const lm5 = landmarks[5];
  const lm17 = landmarks[17];

  // MediaPipe coordinates are normalized [0.0 - 1.0]
  // Scale to viewport pixels
  const u5 = lm5.x * activeViewport.width;
  const v5 = lm5.y * activeViewport.height;

  const u17 = lm17.x * activeViewport.width;
  const v17 = lm17.y * activeViewport.height;

  if (!xrDepthInfo) return;

  // Retrieve physical depth in meters at the coordinates
  const depth5 = xrDepthInfo.getDepthInMeters(lm5.x, lm5.y);
  const depth17 = xrDepthInfo.getDepthInMeters(lm17.x, lm17.y);

  if (depth5 === 0 || depth17 === 0) {
    updateUIState("Calibration error: Keep hand flat and close to surface.");
    return;
  }

  // Unproject to 3D world space
  const p5_3d = unproject(u5, v5, depth5, activeViewport, activeProjectionMatrix);
  const p17_3d = unproject(u17, v17, depth17, activeViewport, activeProjectionMat

  // Compute knuckle physical width (mm)
  const physicalWidth = calculateDistance(p5_3d, p17_3d) * 1000;

  // Clean signal with Kalman filter
  filterMeasurement(physicalWidth);
}

// Matrix unprojection implementation

```

```

function unproject(u, v, depth, viewport, projectionMatrix) {
  const invProj = mat4.create();
  mat4.invert(invProj, projectionMatrix);

  // Normalized Device Coordinates (NDC)
  const ndcX = (2 * u) / viewport.width - 1;
  const ndcY = 1 - (2 * v) / viewport.height;
  const ndcZ = 2 * depth - 1;

  const vec = vec4.fromValues(ndcX, ndcY, ndcZ, 1.0);
  vec4.transformMat4(vec, vec, invProj);

  // Homogeneous coordinates division to retrieve true physical meters
  return vec3.fromValues(
    vec[0] / vec[3],
    vec[1] / vec[3],
    vec[2] / vec[3]
  );
}

function calculateDistance(p1, p2) {
  return Math.sqrt(
    Math.pow(p1[0] - p2[0], 2) +
    Math.pow(p1[1] - p2[1], 2) +
    Math.pow(p1[2] - p2[2], 2)
  );
}

```

6. Defensive Stabilization & Signal Filtering

A single pixel jitter in a noisy video stream can lead to an incorrect bangle recommendation. Implement a **1D Kalman Filter** to smooth raw measurements.

```

class KalmanFilter {
  constructor(processNoise = 0.005, measurementNoise = 0.5, estimationError = 1.0) {
    this.q = processNoise; // Process noise covariance
    this.r = measurementNoise; // Measurement noise covariance
    this.p = estimationError; // Estimation error covariance
    this.x = initialValue; // Value estimate
    this.k = 0; // Kalman gain
  }

  update(measurement) {
    // Prediction Update
    this.p = this.p + this.q;

    // Measurement Update
    this.k = this.p / (this.p + this.r);
    this.x = this.x + this.k * (measurement - this.x);
    this.p = (1 - this.k) * this.p;

    return this.x;
  }
}

const filter = new KalmanFilter();
let stableMeasurementCount = 0;
const REQUIRED_STABLE_FRAMES = 45; // ~1.5 seconds of static hand tracking

```

```
function filterMeasurement(rawVal) {
  // Extreme outlier validation
  if (rawVal < 45 || rawVal > 85) return;

  const smoothedVal = filter.update(rawVal);
  const variance = Math.abs(rawVal - smoothedVal);

  if (variance < 0.8) {
    stableMeasurementCount++;
  } else {
    // Reset if user is moving their hand
    stableMeasurementCount = 0;
  }

  updateCalibrationProgress(stableMeasurementCount / REQUIRED_STABLE_FRAMES);

  if (stableMeasurementCount >= REQUIRED_STABLE_FRAMES) {
    displayFinalBangleSize(smoothedVal);
  }
}
```

7. Bangle Sizing Lookup Map

Bangles are traditionally sold in sizes that signify their internal diameter in inches and sixteenths of an inch.

- Size 2.2 → 2 inches and 2/16 inches = 2.125 in $\approx$ 54.0 mm
- Size 2.4 → 2 inches and 4/16 inches = 2.250 in $\approx$ 57.2 mm
- Size 2.6 → 2 inches and 6/16 inches = 2.375 in $\approx$ 60.3 mm
- Size 2.8 → 2 inches and 8/16 inches = 2.500 in $\approx$ 63.5 mm
- Size 2.10 → 2 inches and 10/16 inches = 2.625 in $\approx$ 66.7 mm

Bangle Size Algorithm

```
const BANGLE_SIZES = [
  { size: "2.2", diameterMM: 54.0 },
  { size: "2.4", diameterMM: 57.2 },
  { size: "2.6", diameterMM: 60.3 },
  { size: "2.8", diameterMM: 63.5 },
  { size: "2.10", diameterMM: 66.7 }
];

function getRecommendedBangleSize(measuredKnuckleWidth) {
  // Add a 2mm tolerance factor for physical hand compression
  const searchDiameter = measuredKnuckleWidth - 2.0;

  for (let option of BANGLE_SIZES) {
    if (searchDiameter <= option.diameterMM) {
      return option;
    }
  }
  // Fallback for larger sizes
  return { size: "2.12+", diameterMM: 69.8 };
}
```

}

8. Sizing Agent's Testing Checklist

To confirm correct implementation, run through this execution sequence:

- ☐ **Is WebXR loading?** Verify the "Start AR Sizer" button loads without throwing JavaScript browser support errors.
- ☐ **Check Depth Sensing:** Ensure `frame.getDepthInformation(view)` returns a non-null object when aiming at a surface.
- ☐ **Address Camera Lockout:** Double-check that `getUserMedia` is **not** called anywhere, and that frames are extracted directly from the WebGL rendering pipeline.
- ☐ **Test Hand Proximity:** Verify that moving the hand closer or further from the camera does not dramatically change the calculated physical size. The values should remain stable within